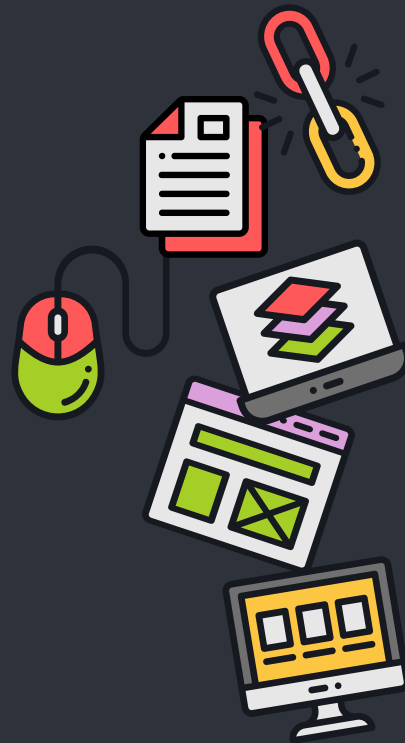


```
1  
2  
3  
4  
5 import astropy  
6  
7  
8  
9  
10  
11  
12  
13  
14
```



intro2astro.py

contents.json

Contents of This Lesson:

```
{  
  What is astropy  
  Subpackages  
  Units Quantities and Constants  
  Observation Planning  
  Inputs and Outputs  
}
```

Maria Vincent

```
1
2
3
4
5
6 class what_is_astropy:
7
8     def __init__(self):
9         return definition
10
11
12
13
14
```



Astropy is a Python package that offers tools and functions that are useful for various tasks in astronomy and astrophysics, including and not limited to planning an observation, reducing the data from the observation, analysing it, and other numerical and modeling tasks.

```
1
2
3
4
5
6 class install_&_import:
7
8     def __init__(self):
9         return definition
10
11
12
13
14
```

```
1  # Using pip, the Python package installer
2
3  pip install astropy
4
5
6
7
8
9
10
11
12
13
14
```

```
1  # Using conda
```

```
2  
3  conda install astropy
```

```
4  
5  # since it comes with Anaconda, to update:
```

```
6  
7  conda update astropy
```

```
1  # Using conda
2
3  conda install astropy
4
5  # since it comes with Anaconda, to update:
6
7  conda update astropy
8
9  # to include the recommended/all available
10 dependencies with installation
11
12 conda install --channel conda-forge --channel
13 defaults scipy matplotlib
14
```



```
1
2
3
4
5
6 class sub_and_other_packages:
7
8     def __init__(self):
9         return list_of_possibilities
10
11
12
13
14
```

```
1  from astropy import subpackage
2  import astropy.subpackage as s
3  from astropy.subpackage import class
4
5  """
6
7  Most of astropy's functionalities lie in
8  subpackages, that can be imported as above, or
9  using shortcuts, or a class alone can be
10 imported from a subpackage. Other packages are
11 often used with or besides these subpackages
12
13 """
14
```

Subpackages

units	Handles defining, converting between, and performing arithmetic with physical quantities
constants	Contains a number of physical constants useful in Astronomy
coordinates	Contains classes for celestial/spatial coordinates and their velocity components, and tools to uniformly convert between common coordinate systems.

Coordinated packages

astroquery	Tools for querying online astronomical data sources.
ccdproc	Package to do basic CCD data reduction.
photutils	Photometry and related image-processing tools.

Affiliated packages

astroML	Tools for machine learning and data mining in Astronomy.
astroplan	An open source Python package to help astronomers plan observations.
dust_extinction	Interstellar dust extinction curves

```
1
2
3
4
5
6 class quantities:
7
8     def __init__(self):
9         return list_of_possibilities
10
11
12
13
14
```

```
1  import astropy.units as u
2
3
4  """
5      Helpful when handling and calculating using
6      different quantities with different units
7      """
8
9
10
11
12
13
14
```

```
1  import astropy.units as u
2
3
4  q = 15.1 * u.meter / (32.0 * u.second)
5  print(q.value)
6  print(q.unit)
7
8
9
10
11
12
13
14
```



```
1  import astropy.units as u
2
3
4  q = 15.1 * u.meter / (32.0 * u.second)
5  print(q.value)
6  print(q.unit)
7
8
```

```
0.471875
m / s
```

```
1  import astropy.units as u
2
3
4  x = 1.0 * u.parsec
5  x.to(u.km)
6
7
8
9
10
11
12
13
14
```

```
1 import astropy.units as u
2
3
4 x = 1.0 * u.parsec
5 x.to(u.km)
6
7
8
```

```
<Quantity 3.08567758e+13 km>
```

```
1  import astropy.constants as c
2
3
4  """
5
6  A library of the oft-used constants in
7  astronomy, to avoid having to create
8  variables
9  """
10
11
12
13
14
```

```
1  import astropy.constants as c
2
3
4  print(c.G)
5
6
7
8
9
10
11
12
13
14
```

```
1  import astropy.constants as c
2
3
4  print(c.G)
5
6
7
```

```
Name      = Gravitational constant
Value     = 6.6743e-11
Uncertainty = 1.5e-15
Unit      = m3 / (kg s2)
Reference = CODATA 2018
```

```
1  import astropy.units as u
2  import astropy.constants as c
3
4
5
6  Calculate the gravitational force between a 3
7  solar mass star and its 10 Jupiter mass planet
8  2 au away.
9
10
11
12
13
14
```

ASSIGNMENT

ASSIGNMENT

intro2astro.py

quantities.py

```
1 import astropy.units as u
2 import astropy.constants as c
3
4 m_star = 3 * c.M_sun
5 m_planet = 10 * c.M_jup
6 distance = 2 * u.au
7 F = c.G * m_star * m_planet / distance**2
8 print(F)
```

1.889285600523636e+48 kg m³ / (AU² s²)


```
1
2
3
4
5
6 class input_out:
7
8     def __init__(self):
9         return i/o
10
11
12
13
14
```

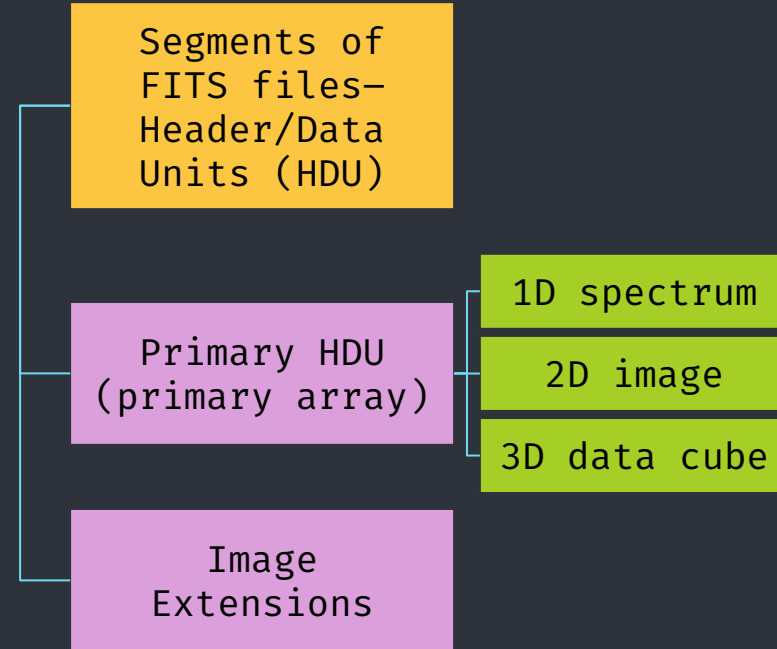
1
2
3
4
5
6
7
8
9
10
11
12
13
14

A unified interface for
reading and writing
data in different
formats

```
1  from astropy.table import Table
2
3
4  t = Table.read('photometry.dat', format='ascii.daophot')
5  filename = 'photometry_latex.tex'
6  t.write(filename, format='latex')
7
8
9
10
11
12
13
14
```

```
1  FITS (Flexible Image Transport
2  System) is the data format most
3  widely used within astronomy for
4  transporting, analyzing, and
5  archiving scientific data files.
6  FITS is much more than just
7  another image format (such as JPG
8  or GIF) and is primarily designed
9  to store scientific data sets
10 consisting of multidimensional
11 arrays (images) and 2-dimensional
12 tables organized into rows and
13 columns of information
14
```

```
1  FITS (Flexible Image Transport
2  System) is the data format most
3  widely used within astronomy for
4  transporting, analyzing, and
5  archiving scientific data files.
6  FITS is much more than just
7  another image format (such as JPG
8  or GIF) and is primarily designed
9  to store scientific data sets
10 consisting of multidimensional
11 arrays (images) and 2-dimensional
12 tables organized into rows and
13 columns of information
14
```



```
1  FITS (Flexible Image Transport
2  System) is the data format most
3  widely used within astronomy for
4  transporting, analyzing, and
5  archiving scientific data files.
6  FITS is much more than just
7  another image format (such as JPG
8  or GIF) and is primarily designed
9  to store scientific data sets
10 consisting of multidimensional
11 arrays (images) and 2-dimensional
12 tables organized into rows and
13 columns of information
14
```

Segments of
FITS files–
Header/Data
Units (HDU)

Header Unit

a sequence of
fixed-length 80-character
keyword records

Data Unit

if present, immediately
follows the last 2880-byte
block in the header unit
as is a value or comment
for the keyword

```
1  from astropy.io import fits
2  fits_img_fn =
3  fits.util.get_testdata_filepath('test0.fits')
4  # Data that came with astropy installation
5  hdul = fits.open(fits_img_fn) # header data unit (HDU)
6  list
7  hdul.info()
8
9
10
11
12
13
14
```

ASSIGNMENT

```
1
2
3
4
5
6 class observations:
7
8     def __init__(self):
9         return planning_observations
10
11
12
13
14
```


1
2
3
4
5
6
7
8
9
10
11
12
13
14

Plan observations
taking into account the
targets, dates and
times, location of
observing, moon

```
1 from astropy.coordinates import SkyCoord, EarthLocation,  
2 AltAz  
3 from astroplan import Observer
```

```
4  
5 You have to plan an observation of Fomalhaut  
6 from Subaru. The first step is to generate the  
7 coordinates of the star and the observing site.  
8 The packages here are a hint
```

```
9  
10 First:
```

```
11  
12 !pip install astroplan  
13  
14
```

Don't peek at the
next page until after
trying this out

ASSIGNMENT

ASSIGNMENT

intro2astro.py

observations.py

```

1  from astropy.coordinates import SkyCoord, EarthLocation,
2  AltAz
3  from astrophysics import Observer
4  fomalhaut = SkyCoord.from_name('Fomalhaut')
5  subaru = Observer.at_site('Subaru', timezone='US/Hawaii')
6  subaru_loc = EarthLocation(lat=subaru.latitude,
7  lon=subaru.longitude)
8  print(f'Sky Coords of Fomalhaut: {fomalhaut} and Geocentric
9  Coords of Subaru: {subaru_loc}')
```

```

Sky Coords of Fomalhaut: <SkyCoord (ICRS): (ra, dec) in deg
(344.41269272, -29.62223703)> and Geocentric Coords of
Subaru: (-5460925.6854608, -2491437.49064199, 2149539.83199706)
m
```

```
1
2
3      *Exercise for the attendees*:
4      There are other tools you
5      need like astropy.time and
6      astropy.visualization to
7      plot and visualize the
8      night-time observability
9
10
11
12
```

```
13      *Because the presenter may have been tired...
14
```

OPTIONAL
ASSIGNMENT

```
1
2
3
4
5
6 class questions:
7
8     def __init__(self):
9         return answers
10
11
12
13
14
```

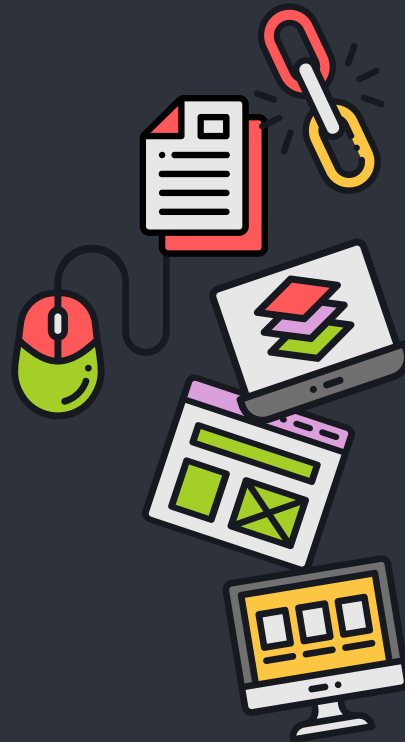


References:

- https://philuttley.github.io/prog4aa_lesson2/09-astropyintro/index.html
- <https://www.astropy.org/affiliated/index.html>
- <https://docs.astropy.org/en/stable/units/>
- <https://fits.gsfc.nasa.gov/fitsprimer.html>

1
2
3
4
5
6
7
8
9
10
11
12
13
14

```
import scipy
```



intro2astro.py

contents.json

Contents of This Lesson:

```
{  
  What is scipy  
  Units Quantities and Constants  
  Optimizer  
  Interpolation  
  Q&A  
}
```

Maria Vincent


```
1
2
3
4
5
6 class what_is_scipy:
7
8     def __init__(self):
9         | return definition
10
11
12
13
14
```

1
2
3
4
5
6
7
8
9
10
11
12
13
14



Scipy is a Python package that offers
Fundamental algorithms for scientific
computing in Python, including and not
limited to optimization, interpolation,
matrix manipulation and statistics.

```
1
2
3
4
5
6 class install_&_import:
7
8     def __init__(self):
9         return definition
10
11
12
13
14
```

```
1  # Using pip, the Python package installer
2
3  pip install scipy
4
5
6
7
8
9
10
11
12
13
14
```

```
1  # Using pip, the Python package installer
2
3  pip install scipy
4
5  # Using conda
6
7  conda install scipy
8
9  # since it comes with Anaconda, to update:
10
11  conda update scipy
12
13
14
```

```
1
2
3
4
5
6 class quantities:
7
8     def __init__(self):
9         return list_of_possibilities
10
11
12
13
14
```

```
1  from scipy import constants as c
2
3
4  """
5
6  A set of built-in scientific constants and
7  unit factors
8  """
9
10
11
12
13
14
```

```
1  from scipy import constants as c
2
3
4  print(c.pi)
5
6
7
8
9
10
11
12
13
14
```



```
1  from scipy import constants as c
2
3
4  print(c.pi)
5
6
7
8
```

3.141592653589793

```
1  from scipy import constants as c
2
3  print(dir(c))
4
5
6
7
8
9
10
11
12
13
14
```

```
1 from scipy import constants as c
2
3 print(dir(c))
4
```

```
['Avogadro', 'Boltzmann', 'Btu', 'Btu_IT', 'Btu_th', 'ConstantWarning', 'G', 'Julian_year', 'N_A', 'Planck',
'R', 'Rydberg', 'Stefan_Boltzmann', 'Wien', '__all__', '__builtins__', '__cached__', '__doc__', '__file__',
'__loader__', '__name__', '__package__', '__path__', '__spec__', '_codata', '_constants', '_obsolete_constants',
'acre', 'alpha', 'angstrom', 'arcmin', 'arcminute', 'arcsec', 'arcsecond', 'astronomical_unit', 'atm',
'atmosphere', 'atomic_mass', 'atto', 'au', 'bar', 'barrel', 'bbl', 'blob', 'c', 'calorie', 'calorie_IT',
'calorie_th', 'carat', 'centi', 'codata', 'constants', 'convert_temperature', 'day', 'deci', 'degree',
'degree_Fahrenheit', 'deka', 'dyn', 'dyne', 'e', 'eV', 'electron_mass', 'electron_volt', 'elementary_charge',
'epsilon_0', 'erg', 'exa', 'exbi', 'femto', 'fermi', 'find', 'fine_structure', 'fluid_ounce', 'fluid_ounce_US',
'fluid_ounce_imp', 'foot', 'g', 'gallon', 'gallon_US', 'gallon_imp', 'gas_constant', 'gibi', 'giga', 'golden',
'golden_ratio', 'grain', 'gram', 'gravitational_constant', 'h', 'hbar', 'hectare', 'hecto', 'horsepower', 'hour',
'hp', 'inch', 'k', 'kgf', 'kibi', 'kilo', 'kilogram_force', 'kmh', 'knot', 'lambda2nu', 'lb', 'lbf',
'light_year', 'liter', 'litre', 'long_ton', 'm_e', 'm_n', 'm_p', 'm_u', 'mach', 'mebi', 'mega', 'metric_ton',
'micro', 'micron', 'mil', 'mile', 'milli', 'minute', 'mmHg', 'mph', 'mu_0', 'nano', 'nautical_mile',
'neutron_mass', 'nu2lambda', 'ounce', 'oz', 'parsec', 'pebi', 'peta', 'physical_constants', 'pi', 'pico',
'point', 'pound', 'pound_force', 'precision', 'proton_mass', 'psi', 'pt', 'queto', 'quetta', 'ronna', 'ronto',
'short_ton', 'sigma', 'slinch', 'slug', 'speed_of_light', 'speed_of_sound', 'stone', 'survey_foot',
'survey_mile', 'tebi', 'tera', 'test', 'ton_TNT', 'torr', 'troy_ounce', 'troy_pound', 'u', 'unit', 'value',
'week', 'yard', 'year', 'yobi', 'yocto', 'yotta', 'zebi', 'zepto', 'zero_Celsius', 'zetta']
```

1
2
3
4
5
6
7
8
9
10
11
12
13
14



Metric	Pressure
Binary	Volume
Mass	Speed
Angle	Temperature
Time	Energy
Length	Power
Force	

1
2
3
4
5
6
7
8
9
10
11
12
13
14



kilo: 1000

centi: 0.01

micro: 1e-06

Metric

Pressure

Binary

Volume

Mass

Speed

Angle

Temperature

Time

Energy

Length

Power

Force

1
2
3
4
5
6
7
8
9
10
11
12
13
14



degree:
0.017453292519943295

arcmin (arcminute):
0.0002908882086657216

arcsec (arcsecond):
4.84813681109536e-06

Metric	Pressure
Binary	Volume
Mass	Speed
Angle	Temperature
Time	Energy
Length	Power
Force	

```
1
2
3
4
5
6 class optimizers:
7
8     def __init__(self):
9         return optimizer_function
10
11
12
13
14
```

```
1  from scipy import optimizers as opt
2
3
4  """
5
6  Optimizers are a set of procedures defined in
7  SciPy that find the minimum value of a
8  function or the root of an equation, or fit
9  an equation.
10 """
11
12
13
14
```



```
1  from scipy.optimize import root
2  import numpy as np
3
4  def eqn(x):
5      return x + np.cos(x)
6  myroot = root(eqn, 0)
7  print(myroot.x)
8
9
10
11
12
13
14
```

finds the
root of an
equation

```
1 from scipy.optimize import root
2 from numpy import np
3
4 def eqn(x):
5     return x + np.cos(x)
6 myroot = root(eqn, 0)
7 print(myroot.x)
```

```
[-0.73908513]
```

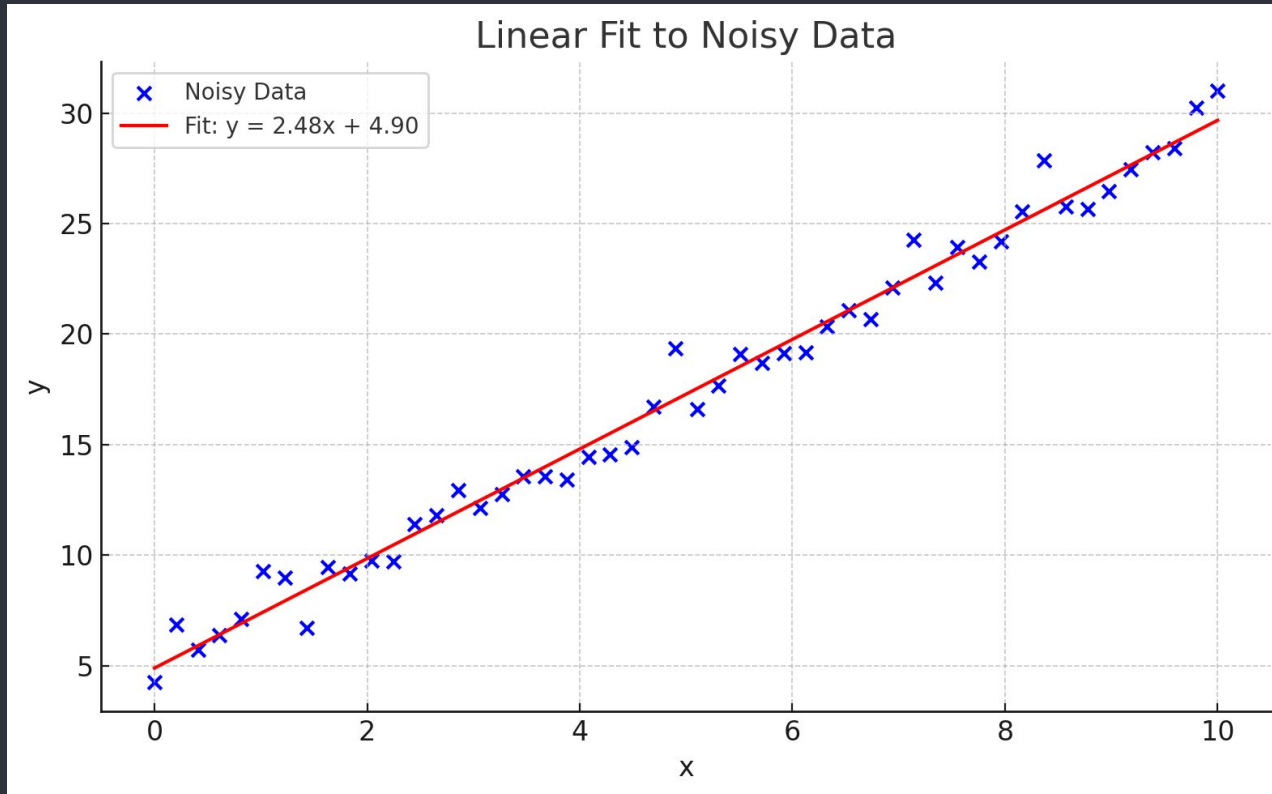
```
1  from scipy.optimize import minimize
2
3
4  def eqn(x):
5      return x**2 + x + 2
6  my_min = minimize(eqn, 0)
7  print(my_min.x)
8
9
10
11
12
13
14
```

finds the
minimum of an
equation

```
1  from scipy.optimize import minimize
2
3
4  def eqn(x):
5      return x**2 + x + 2
6  my_min = minimize(eqn, 0)
7  print(my_min.x)
8
9
```

```
array([-0.50000001])
```

```
1  from scipy.optimize import curve_fit
2  import numpy as np
3  import matplotlib.pyplot as plt
4
5  def eqn(x, m, c):
6      return m*x + c
7  # define the real data
8  x_data = np.linspace(0, 10, 50)
9  m, c = 2.5, 5.0
10 y_data = eqn(x_data, m, c) + np.random.normal(0, 1, 50)
11 # fit a line; popt: optimised parameters; pcov: covariance matrix
12 popt, pcov = curve_fit(eqn, x_data, y_data) # add param: guesses
13 m_fit, c_fit = popt[0], popt[1]
14 # plot: x_data vs y_data; x_data vs eqn(x_data, m_fit, c_fit)
```



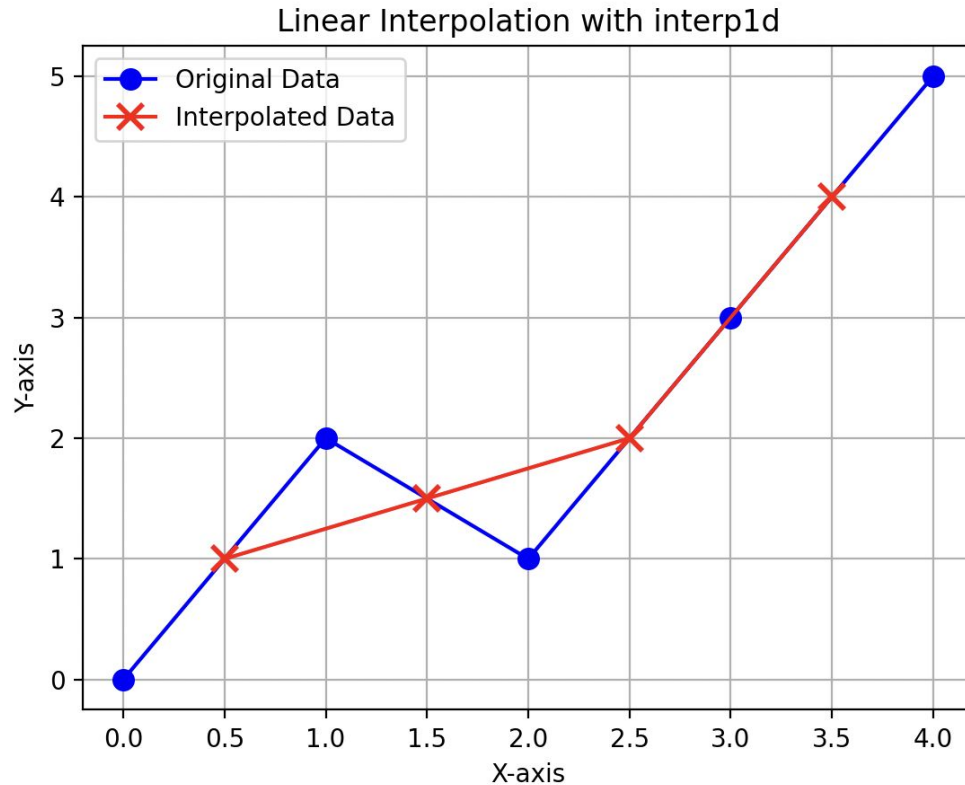
```
1
2
3
4
5
6 class interpolation:
7
8     def __init__(self):
9         return interp
10
11
12
13
14
```

```
1  from scipy import interpolate
2
3
4  """
5
6  Interpolation: A method for generating points
7  between given points.
8  """
9
10
11
12
13
14
```



```
1  from scipy.interpolate import interp1d
2  import numpy as np
3  import matplotlib.pyplot as plt
4
5  # define sample data
6  x_data = np.array([0, 1, 2, 3, 4])
7  y_data = np.array([0, 2, 1, 3, 4])
8  # Create the interpolation function (defaults to linear interp)
9  f_interp = interp1d(x_data, y_data)
10 # define new x
11 x_new = np.array([0.5, 1.5, 2.5, 3.5])
12 # get y from f_interp
13 y_interp = f_interp(x_new)
14 # plot: x_data vs y_data; x_new vs y_interp
```

interpolate a
dataset with
1 variable.



```
1  from scipy.interpolate import
2  RegularGridInterpolator, RectBivariateSpline
3  import numpy as np
4  import matplotlib.pyplot as plt
```

ASSIGNMENT: Try 2D interpolation; i.e. interpolate data that is a 2D array

Use

<https://docs.scipy.org/doc/scipy/tutorial/interpolate/>



What I hope you learnt:

- The existence of astropy and scipy
- Some of the tools available in astropy for astronomical math, observation planning, and data analysis
- How to wield these tools



```
1
2
3
4
5
6 class questions:
7
8     def __init__(self):
9         return answers
10
11
12
13
14
```